

Sorokin School



Task-Management System [часть 2]

🔥 Задание к модулю №2 по интенсиву Spring Boot

Вы расширяете свой проект, начатый в прошлом модуле, до полноценного CRUD-приложения для сущности Task.

Описание
В задании вам предстоит реализовать эндпоинты для создания, получения, обновления и удаления задач. При этом хранение данных осуществляется в сервисе с использованием in-memory хранилища на базе `HashMap`.

- Знания для выполнения (что вам предстоит закрепить)**
- Работа с различными HTTP-методами (GET, POST, PUT, DELETE).
 - Реализация многослойной архитектуры приложения, где контроллеры делегируют бизнес-логику сервису.
 - Организация хранения данных в памяти с использованием коллекций Java (например, `HashMap`).
 - Правильное использование HTTP-статусов (например, 201 Created, 404 Not Found).

Задание на разработку
🔴 Что конкретно детально нужно реализовать

Доработайте контроллер `TaskController`:

- Обновите класс `TaskController`.
- Создайте следующие эндпоинты:
 - `GET /tasks` – для получения списка всех задач.
 - `GET /tasks/{id}` – для получения задачи по id.
 - `POST /tasks` – для создания новой задачи.
 - `PUT /tasks/{id}` – для обновления задачи.
 - `DELETE /tasks/{id}` – для удаления задачи.
- Обеспечьте корректное использование HTTP-статусов:
 - 200 Ok при успешной обработке
 - 201 Created при успешном создании.
 - 404 Not Found, если задача с указанным id не найдена (просто отлавливайте `NoSuchElementException` из сервисного слоя)

Доработайте сервис `TaskService`:

- В сервисе организуйте хранение задач с помощью `HashMapLong, Task`.
- Реализуйте логику генерации уникальных идентификаторов (например, с использованием счетчика).
- **Добавьте методы для:**
 - Создания новой задачи
 - При создании задачи вы должны проверять, что входящие данные не содержат id и status.
 - Эти поля устанавливаются сервисом: генерируется уникальный id и статус автоматически проставляется как `"CREATED"`.
 - Получения списка всех задач
 - Получения задачи по id
 - Обновления существующей задачи (PUT).
 - Если задача находится в статусе `DONE`, редактирование должно блокироваться с возвратом ошибки.
 - Можно предусмотреть возможность перевода задачи из `DONE` обратно в `IN_PROGRESS` для возобновления работы.
 - Удаления задачи по id

Логирование:

- Добавьте логирование вызовов методов контроллера: логируйте входные параметры, начало и окончание обработки запроса, а также случаи возникновения ошибок.

Советы по реализации
Немного заметок о том как лучше реализовать задание

- **Четкое разделение ответственности:**
Контроллер должен принимать HTTP-запросы и делегировать обработку сервису, а весь функционал CRUD-операций реализуйте в `TaskService`.
- **Работа с `HashMap`:**
Обратите внимание на правильное управление ключами (id). Для генерации новых id можно использовать простой счетчик, который увеличивается при каждом создании задачи.
- **Базовая обработка ошибок:**
Проверьте, что если запрашиваемая задача отсутствует, вы возвращаете корректный HTTP-статус (например, 404 Not Found) с понятным сообщением.
- **Логирование:**
Используйте логирование, чтобы фиксировать начало обработки запроса, входные данные и результат выполнения. Это поможет вам отладить приложение и понять, как обрабатываются запросы.

Тестирование

Проверка запуска приложения:
Запустите приложение и убедитесь, что оно стартует без ошибок.

Проверьте работу эндпоинтов с помощью Postman:

- **GET /tasks:**
Отправьте запрос для получения списка задач и убедитесь, что возвращается список, который включает задачи, предопределенные в предыдущем задании или созданные на этом этапе.